

新生培训讲义

四足机器人本体硬件平台，URDF的讲解和导出方式

我们先来初步讲解一下四足机器人的结构，目前最主流的四足机器人构型和算法，它们的始祖是mit mini cheetah

颠覆了以往采用高减速比关节方案，开创性地采用了准直驱电机方案，外转子无刷电机加上单级行星齿轮减速大约是(6比1)，在保证力矩不是特别小的情况下，让整个力矩的控制更加精准。包括后续人型机器人的发展也是借助了准直驱电机和强化学习，得以有了今天的运动控制发展。因为做仿生足式机器人其实构型是逐渐收敛的，且了解构型再控制肯定是最优的过程，单条腿是有三个自由度，第一个自由度是和身体贴的很近，带着整个腿去做外摆和内收的动作，称为髋关节，然后经过一个杆件直接和大腿电机固定，大腿关节就是控制整个大腿杆件摆动，在四足机器人运动里面主要发挥着将足端往前或向后移的作用。然后小腿电机则是通过一个平行四边形连杆带动着小腿，确保小腿电机转动和连杆转动的角度是1比1的。有时候会在这个位置添加一个减速器，然后再通过连杆带动，主要目的是避免腿部质心离躯干太远，导致后续会讲到的单刚体动力学不适用，同时也是让髋关节电机和大腿电机的力矩更小，控制更稳定，然后小腿在整个四足机器人运动的过程中就是主要发挥着支撑机身质量，提供反作用力的作用，一般需要的力会相比于髋关节和大腿关节更大。

针对去年障碍赛机器狗做一下设备拆解。

关节电机:mit混合控制



GO Motor

Model GO-M8010-6	Weight About 530g
Maximum Current 40A	Maximum Rotational Speed 30rad/s (@24VDC)
Reduction Ratio 1:6.33	Size 96.5x92.5x42.3mm
Maximum Torque 23.7NM	Torque Constant 0.63895Nm/A
Communication Mode RS-485	Communication Baud Rate 4Mbps
Communication Control Frequency 6000Hz	Motor Encoder Resolution 15bit
Operation Environment -5°C ~ 40°C	Working Voltage 12V~30VDC 24VDC recommend
Motor Perceptual Feedback Torque, Angle, Angular Velocity, Angular Acceleration, Temperature	
Motor Control Instruction Torque, Angle, Angular Velocity, Stiffness, Damping	



* This product is a civilian robot. We kindly request that all users refrain from making any dangerous modifications or using the robot in a hazardous manner.
 * Please visit Untree Robotics Website for more related terms and policies, and comply with local laws and regulations.



MIT 混合控制模式将传统的**前馈力矩控制**与**底层闭环阻抗/PD 控制**在驱动器端（FOC 固件层）直接融合。每个执行周期，上位机向电机下发 5 个维度的控制参数，电机驱动器在 10kHz ~ 20kHz 的高频电流环上执行以下控制律：

$$\tau_{\text{motor}} = \tau_{\text{ff}} + K_p(q_{\text{des}} - q) + K_d(\dot{q}_{\text{des}} - \dot{q})$$

其中下发的 5 个控制量分别为：

- q_{des} ：期望关节角度（Target Position, rad）
- $\dot{q}_{\text{des}}$ ：期望关节角速度（Target Velocity, rad/s）
- K_p ：关节刚度/比例增益（Joint Stiffness, N · m/rad）
- K_d ：关节阻尼/微分增益（Joint Damping, N · m · s/rad）
- τ_{ff} ：前馈力矩（Feedforward Torque, N · m）

是目前使用最多的关节电机控制方式，等式左边是电机最后要输出的力矩，然后等式右边按顺序就是前馈力矩，也就是人为添加的一个补偿量，因为单纯靠右边第二和第三个PD控制，会有因为重力或者摩擦力造成的稳态误差，如果你需要的话，就能添加一个前馈力矩去补偿。然后PD控制就可以理解成一个带阻尼器的弹簧，期望位置就是平衡点，如果远离期望位置就会给一个线性力拉回期望位置，具体这个拉回的刚度有多强烈，就是KP的大小去影响的。然后KD就是理解成阻尼器，如果期望速度是0，那么上面这个式子就会尽量让电机的速度拉回到0。前面提到的期望值都是可以随意设置的。可以

注意到，如果把前馈力矩置成0，就成了PD控制，把前馈力矩和kd置成0，就成了位置控制，把前馈力矩和KP置成0，就成了速度控制，我们四足控制，传统控制里面MPC就只会用前馈力矩，然后MPC+WBC就只会用5个参数。然后强化学习一般就会用KP KD 和Qdes,速度期望和前馈力矩是设计成0的。

然后给电机发消息的话，其实就是把这5个参数打包成字节然后通过信号线发送进去。然后电机内部的芯片就运行然后开始驱动电机，

类型(type)	位(bit)	符号	说明	值
包头 (2Byte)	0-15 (2Byte)	HEAD	数据包头部	0xFE 0xEE
模式设置 (1Byte)	16-19 [4bit]	ID	目标电机 ID	0,1,2,3 ... 13,14 15.表示向所有电机广播数据 (此时总线无返回值)
	20-22 [3bit]	STATUS	电机工作模式	0.锁定(Default) 1.FOC 闭环 2.编码器校准(等待 5s, 期间不可以给电机发送任何数据包) 3-7. 保留
	23 [1bit]	保留		
控制参数 (12Byte)	24-39 (2Byte)	τ_{set}	期望电机转矩	$\tau_{ff} = 0.75 \text{ (N.m)}$ $\tau_{set} = \tau_{ff} * 256$ $= 192$ $ T_{ff} \leq 127.99N.m$
	40-55 (2Byte)	ω_{set}	期望电机速度	$\omega_{des} = 3.14159 \text{ (rad/s)}$ $\omega_{set} = \frac{\omega_{des}}{2\pi} * 256$ $= 128$ 注: $2\pi/s = 6.28rad/s = 60RPM$ $ \omega_{des} \leq 804.0 \text{ rad/s}$
	56-87 (4Byte)	θ_{set}	期望电机输出位置(多圈累加)	$\theta_{des} = 90^\circ = \frac{\pi}{2} = 1.57 \text{ (rad)}$ $\theta_{set} = \frac{\theta_{des}}{2\pi} * 32768$ $= 8187$ 注: $2\pi = 360^\circ = 6.2831rad$ $ \theta_{des} \leq 411774 \text{ rad}(\approx 65535 \text{ 圈})$
	88-103 (2Byte)	K_{pos}	电机刚度系数/ 位置误差比例系数	$K_p = 0.1$ $K_{pos} = K_p * 1280$ $= 128$ 注: $0 \leq K_p \leq 25.599$
	104-119 (2Byte)	K_{spd}	电机阻尼系数/ 速度误差比例系数	$K_w = 0.2$ $K_{spd} = K_w * 1280$ $= 256$ 注: $0 \leq K_w \leq 25.599$
校验部分 (2Byte)	120-135 (2Byte)	CRC16	CRC16 校验结果	CRC16_CCITT 多项式计算 0-119 位数据的结果
共计:	17 Byte			

至于更底层电机驱动底层的事，就要看厂家的电机产品质量了。

关节电机除了是一个执行器，其实还是一个传感器，很多电机都是你给它发一个消息然后它就给你返还一次它的状态，它会给我们反馈的数据有电机位置，电机速度，力矩。这些传感信息如果处理得当会有很大用处。

imu



像超核的产品是已经高度封装了的，将它与机载电脑通过usb连接，然后使用usb转uart的驱动就能获取到imu的数据，正常的imu是只能直接获取三维角速度和三维的加速度，然后好一点的imu会内置卡

尔曼滤波处理，同时也把姿态角也发送出来。我们用的H14就能直接给出9维数据。

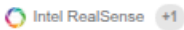
深度相机和激光雷达:我们运控使用深度相机和激光雷达，主要是去做端到端的rl训练，也就是直接将深度相机和激光雷达给到的信息传入网络训练。在传统控制里面会有通过这些观感设备去精准规划落足点，然后控制算法再让足端精准踩到这些落足点，但是这会很难。我们常用的深度相机就是d435i,然后激光雷达就是Livox MID-360





D435i 能获取什么

D435i 本身有 RGB 相机、双目红外深度相机和 6 轴 IMU。官方规格确认它可以同时输出深度、彩色图像以及陀螺仪和加速度计数据。



数据	形式	可以干什么
RGB 彩色图像	每个像素 RGB	目标识别、二维码、物体检测
左/右红外图像	灰度图	双目匹配、低纹理环境分析
Depth 深度图	每个像素一个距离	障碍物距离、台阶高度、地形
三维点云	x, y, z	由深度图反投影得到
三轴角速度	$\omega_x, \omega_y, \omega_z$	相机旋转运动
三轴加速度	a_x, a_y, a_z	相机运动状态
时间戳	timestamp	图像/IMU同步
相机内参	f_x, f_y, c_x, c_y 等	三维重建、PnP
相机间外参	R, t	RGB、Depth、IMU坐标转换

因此 MID-360 可以直接得到：

数据	含义
x, y, z	激光点在雷达坐标系里的位置
distance	障碍物距离
reflectivity	目标表面对激光的反射强度
tag	点的可信度/特征标签
timestamp	每个点的时间
line	激光扫描线信息
gyro XYZ	内置 IMU 三轴角速度
accel XYZ	内置 IMU 三轴加速度

d435i则是用usb线和机载电脑链接，然后用官方写的ros2功能包就能直接获取信息

mid-360则是通过网口和主机链接(这个是信号线)，另外还需要电源线单独供电。同样也是可以用官方写的ros2功能包就能使用。

The image shows a custom electronic module on a black PCB. Key features include:

- Left Side:** A USB Type-C port and a 48V DC input terminal with a yellow cover.
- Top Edge:** A row of six pin headers labeled 1H1, 1H2, 1H3, 1H4, 1H5, and 1H6.
- Bottom Edge:** Four pin headers labeled Valve4, Valve3, Valve2, Valve1, and a Pump header.
- Central Components:** A 3.3V voltage regulator, a 100µF capacitor, and various other electronic components.
- Markings:** A QR code is located on the bottom left, and the text "48V" and "GND" are printed near the input terminal.



以及分电板:给电机和机载电脑,分电的pcb板。然后就是其他的一些降压模块,整个机器狗使用的硬件都还是少且清的。

URDF的介绍

仿真环境怎么知道我们的机器人长什么样,就是靠URDF(机器人描述文件),URDF本质上是一个刚体树,把真实机器人描述给软件。它里面包含的内容为:机器人有哪些刚体,刚体之间怎么连接,质量是多少,惯量是多少,关节怎么转,碰撞体是什么。

下面是四足机器人树,躯干和四个髋关节相连,然后髋关节和髋刚体相连,髋刚体和大腿关节相连,大腿关节和大腿刚体相连,然后大腿刚体和小腿关节相连,小腿关节与小腿刚体连接,刚体就是link,关节就是joint,整体的顺序就是link-joint-link-joint这样循环往复。正常都是会以link结束。然后名字

类似的joint和link，会是joint带动link，比如髋关节带动髋刚体，然后又因为髋刚体后面的部分是和髋刚体连接，所以被一并带动。形象的来说，link就是骨头，joint就是关节



知道了整个joint和link的连接顺序，其实也就知道了整个机器人的几何骨架，后续讲解的就是给这个骨架附加上皮肤和骨架的质量，长度，转动惯量。和碰撞

<visual>

<origin

xyz="0 0 0"

rpy="0 0 0" />

<geometry>

<mesh filename="../meshes/base_link.STL" />

</geometry>

<material

name="">

<color rgba="0.792156862745098 0.819607843137255 0.933333333333333 1" />

</material>

</visual>

比如上面这个可视化就是在相应的骨架上添加一个皮肤。这里面的base.STL就是一个文件，里面就是一个几何模型。上面给的origin是指相对于这个刚体的坐标的距离。

<collision>

<origin xyz="-0.31 0 0.112" rpy="0 0.7 0" />

```

<geometry>
  <box size="0.045 0.1 0.01" />
</geometry>
</collision>

```

这个就是碰撞，它可以直接像上面的可视化一样，直接索引STL文件过来，就把这个三维模型直接当成碰撞体，后续在仿真中，这个碰撞体的边界，就是实际产生碰撞的地方，但是一般情况下STL文件上面的纹理都会比较复杂，会有很多凸起细节，我们为了节省训练里面的碰撞计算，就会选择用胶囊体去代替原来的STL文件。但是关节碰撞部位，比如机器狗的足端这种，就还是使用原来的STL文件和自己建一个贴合的胶囊体都是可以的。注意，visual是给人看的，collision是给物理引擎算的。就是一种极端情况，如果只是visual有问题，但是collision没有问题，你放到仿真器里面做训练或者sim2sim都是没有问题的，只是看着难受一点。

```

<joint
  name="FL_hip_joint"
  type="revolute">
  <origin
    xyz="0.25944 0.075113 0"
    rpy="0 0 0" />
  <parent
    link="base_link" />
  <child
    link="FL_hip" />
  <axis
    xyz="1 0 0" />
  <limit
    lower="-1.01"
    upper="1.01"
    effort="120.0"
    velocity="22.0" />
</joint>

```

joint就是连接上一级刚体和下一级刚体之间的关节，parent就是父级刚体，也就是上一级刚体，child就是子级刚体，也就是下一级刚体，然后origin的含义就是相对于上一级刚体的距离，后续的axis(转动轴)包括下一级刚体的原点坐标系就是建立在这个点上，关于joint的类型

joint 类型	中文理解	自由度	能不能动	典型用途
revolute	有限角度旋转关节	1	能转，但有限位	四足髋/大腿/膝关节
continuous	无限旋转关节	1	可以一直转	轮子、电机转子
prismatic	直线移动关节	1	沿一条轴平移	滑轨、伸缩机构
fixed	固定连接	0	完全不动	相机、雷达、固定结构件
floating	六自由度自由关节	6	XYZ 平移 + RPY 转动	机器人机身相对世界
planar	平面运动关节	3	平面内 2 平移 + 1 转动	很少用于腿式机器人

revolute是最常见的旋转关节，它会定义有可转动角度的上限和下限，正常机器人的腿部关节就会用这样的类型。

continuous是无限旋转关节，一般会用在轮足机器人的轮子上面，方便能做无限圈数的转动。

fixed会比较特殊一点，一般用上它的时候都会是一个特殊的物品，比如相机，足端，它们需要和上一级刚体紧密链接。但是需要把这部分区分出来。就拿足端距离，它本来就是和小腿刚性链接的，为什么不直接算在小腿里面，而是要用一个fixed去划分出来，是因为有时候在训练的时候，你更需要掌握的是足端的信息而不是小腿的信息，把足端剥离出来会更加方便训练

然后limit针对力矩和转速会有一个限制。这个挺重要的，可以在训练的时候就把力矩和速度做到一定的限制，方便后续实现sim2real,本身电机的极限参数就是这个，如果你在训练的时候力矩和转速还会更大，那必然导致sim2real的失败。但最好的应该还要参考官方给的电机性能曲线，每一个转速下都有一个最大力矩。这样设计才会真正合理。

然后这里的axis定义的就是转动轴的方向，想这里的xyz是1 0 0，那么就是绕着x轴旋转。

```
<inertial>
  <origin
    xyz="0.045604 -0.0009 0.020368"
    rpy="0 0 0" />
  <mass
    value="8.8" />
  <inertia
    ixx="0.044044"
    ixy="1.959E-05"
    ixz="-0.026181"
```

```

    iyy="0.13739"
    iyz="0.00030824"
    izz="0.1462" />
</inertial>

```

这个部分就是在描述这个刚体的质量以及转动惯量，需要注意的是，这个转动惯量的具体大小是和坐标系，质心位置紧密相关的。这个origin的含义就是相对于joint的距离。

总得来看这些坐标系建立，就是joint下面出现的相对距离指的是和上一个joint坐标系的距离，然后link下面，出现的任何visual,inertial,collision,它们下面的相对距离相对的就是这个连杆所属joint的坐标系。

稍微带着同学们读一下代码，
然后我们放到robotview里面实践一下。

MPC的运用

mpc是模型预测控制，可以分成模型和预测两个部分
关键点拆分成 模型 预测 最优控制

它是一种最优控制，写出它的代价一般表达式

$$J = (X - X_d)^T S (X - X_d) + \alpha U^T U$$

整个mpc都是基于这个代价函数去使它最小，就像一个开口的二次函数一样，目的是为了求取到那个x能得到最小的y，这里的x就是我们能够控制的输入量，比如电机的力矩，给电机的期望角度，或者再上层一点就是一个类机械臂结构的末端接触力，因为四足机器人每一条腿其实就是一个三自由度的机械臂，如果我们知道期望的末端接触力就能用雅各比矩阵反向求出各个关节的力矩，

然后我们本次讲的mit minicheetah开源方案就是将四条腿的足端触地力作为控制输入x，在控制学里面经常记为u，每个足端触地力都是三维的，x，y，z三个方向，讲到坐标系，目前行业内通用的坐标系方向就是以狗头方向为x正方向，左侧就是y正方向，上侧就是z正方向（讲到这里结合robotview里面的坐标系），回到前面，然后4个足端，每个足端有三维力，则一共是12维的u，

好的，现在我们已经清楚需要重点关注足端触地力对整个机器狗模型的影响，下面就讲到mpc里面的模型，其实就是对机器人进行物理公式建模，mit使用的是单刚体动力学，把机器人想象成一个刚体，力和力矩的作用都是直接作用到这个刚体上，然后用到的是牛顿-欧拉方程，平动部分就是牛顿第二定律，12维足端力与重力共同作用形成了机器狗的三维加速度。然后把它处理成等式左边是加速度的形式。然后就是欧拉方程，这里需要注意的就是力臂的问题，前面涉及到单刚体动力学，最完美的场景是腿的质量占到躯干质量的十分之一，然后整体的质心就可以近似为躯干的质心，此时我们这个力臂就是足端到躯干质心的距离，然后再到，力矩的总和形成了机身的转动。展开这个求导公式，发现就会出现特别复杂的状态量和状态量相乘了，线性的含义应该是一次函数的形式，放到空间状态方程里面就应该是

这里的 x 就是状态量向量，而 w 就是状态量的一部分，后续它和其他部分堆叠起来就会构成完整的 x ，现在出现了 w 与 w 相乘，那它便不再是一个线性系统，我们想办法把他线性化，就直接把这部分非线性的部分直接忽略掉。只保留线性部分。这是想要简化必须做的工作。然后上面是等式建模。接下来就是不等式约束，其实看了这部分的内容就很能意思到，我们一直是基于控制量在做文章，目标就是在等式和不等式中约束12维的足端接触力，比如求解器寻找到一个满足等式的接触力，它还需要将这个力带入不等式约束看它是否满足。(接下来出现一个ppt,上面列出mpc的几个约束)，首先是摩擦锥，正常的应该是一个摩擦圆，但是为了线性化，则使用了摩擦锥。然后就是力必须是正的，因为足端接触力不可能是吸引力，一定是弹力，方向是在地面上方的。接下来就是最大力限制，小于120N。上面则是把整个模型给建模完了。

在前面有提到过状态量 x 的存在，在这个mpc算法中 x 是一个13维的向量，它包含了机身位置3维，机身线速度3维，机身朝向角3维，机身角速度3维，重力加速度1维。注意将重力加速度放在状态量里面只是一种让后续公式更加简便地处理，没有其他特殊含义。这个 x 每一轮周期都会更新，这里详细介绍一下这些状态量是如何得到的。机身朝向角和机身角速度都是能够直接从imu获取，一般都是从imu获取四元数，但是四元数到朝向角就只有一个变换而已。然后机身位置和机身速度则是通过了卡尔曼滤波，卡尔曼滤波我们在这里就不详细展开讲，大家只需要它其实就是在预测值(也就是像速度积分去预测距离这种)和测量值(直接获取参数的值，但是会有误差)之间取得一个均衡值。然后在对于机身位置这个测量值的话就是

足端世界位置：

$$\mathbf{p}_F^W = \mathbf{p}_B^W + R_{WB}\mathbf{p}_{F/B}^B$$

定义：

$$\mathbf{r}_F^W = R_{WB}\mathbf{p}_{F/B}^B$$

于是：

$$\mathbf{p}_B^W = \mathbf{p}_F^W - \mathbf{r}_F^W$$

如果支撑脚不滑：

$$\mathbf{p}_F^W \approx \text{constant}$$

那么两个时刻：

$$\mathbf{p}_{B,k}^W = \mathbf{p}_F^W - \mathbf{r}_{F,k}^W$$

$$\mathbf{p}_{B,k-1}^W = \mathbf{p}_F^W - \mathbf{r}_{F,k-1}^W$$

两式相减：

$$\mathbf{p}_{B,k}^W - \mathbf{p}_{B,k-1}^W = -(\mathbf{r}_{F,k}^W - \mathbf{r}_{F,k-1}^W)$$

所以：

$$\Delta \mathbf{p}_B^W = -\Delta \mathbf{r}_F^W$$

这才是“用支撑脚测机身 x, y ”最本质的公式。

它实际上测的是：

机身相对于固定支撑脚发生了多少位移。

靠正运动学去获取到测量值，它的原理就是上面,当足端触地则假设它不动，那么此时的机身运动的距离就与足端相对于机身运动的距离。然后足端相对于机身的三维距离可以用正运动学公式求解出来，这个部分因为四足的一条腿只有三个自由度，所以计算起来就是简单的长度乘上正余弦夹角。

2. 把源码原式和公式一一对上

源码：

```
</> C++  
  
p->operator()(0)  
    = l3 * s23 + l2 * s2;
```

对应：

$$x = l_3 \sin(q_2 + q_3) + l_2 \sin q_2$$

源码：

```
</> C++  
  
p->operator()(1)  
    = l1 * sideSign * c1  
    + l3 * (s1 * c23)  
    + l2 * c2 * s1;
```

整理：

$$y = s l_1 \cos q_1 + l_3 \sin q_1 \cos(q_2 + q_3) + l_2 \sin q_1 \cos q_2$$

进一步提取：

$$y = s l_1 \cos q_1 + \sin q_1 [l_2 \cos q_2 + l_3 \cos(q_2 + q_3)]$$

源码：

```
</> C++  
  
p->operator()(2)  
    = l1 * sideSign * s1  
    - l3 * (c1 * c23)  
    - l2 * c1 * c2;
```

对应：

$$z = s l_1 \sin q_1 - l_3 \cos q_1 \cos(q_2 + q_3) - l_2 \cos q_1 \cos q_2$$

整理：

$$z = s l_1 \sin q_1 - \cos q_1 [l_2 \cos q_2 + l_3 \cos(q_2 + q_3)]$$

这些就是官方源码里的实际正运动学。  MIT Biomimetics

然后机身距离的估计值就是用获得到的修正后的速度去积分。接下来就来讨论一下这个修正后的速度，也就是状态量里面机身速度。机身速度的估计值就是用足端相对于机身的速度。这部分仍然用雅可比矩阵，已知关节角速度去求末段线速度。这里不展开讲j是怎么得到的，暂时先将它理解成一个每周周期都能通过正运动学解算而求出来的矩阵

$$\dot{x} = J(q)\dot{q}$$

然后机身速度的测量值就是由imu直接给出的加速度积分而来。最后测量值和估计值取个均衡值，作为最终的速度。然后重力加速度是一个固定的值，那么我们的13维状态x的获取就讲明白了。然后状态空间方程需要罗列一下状态求导量和状态，以及输入量之间的关系。这里面用了向量和矩阵，其实就是将我们之前给的等式给堆叠起来。方便后续使用求解库计算

现在终于能看到核心模型了：

$$\dot{\Theta} = R_z^T(\psi)\omega$$

$$\dot{p} = v$$

$$\dot{\omega} = I_W^{-1} \sum_i [r_i]_{\times} f_i$$

$$\dot{v} = \frac{1}{m} \sum_i f_i + e_3 g$$

$$\dot{g} = 0$$

其中：

$$e_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

同时世界坐标系下的惯量近似：

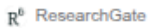
$$I_W \approx R_z(\psi) I_B R_z^T(\psi)$$

其中：

- I_B ：机体系惯量
- I_W ：世界系惯量

于是整个模型就可以写成：

$$\dot{x} = A_c x + B_c u$$

这就是 MIT Convex MPC 最核心的一步。  +1

控制器没法真正连续优化，因为在程序里面都是以2ms为周期去更新状态，没法使用上面的表达式，需要变成

$$x_{k+1} = A_k x_k + B_k u_k$$

则通过扩展系统的状态转移矩阵的到这样的离散模型。比如

$$A_d = e^{A_c \Delta t}$$

最终我们能够得到

$$\boldsymbol{x}_{k+1} = \boldsymbol{A}_{d,k}\boldsymbol{x}_k + \boldsymbol{B}_{d,k}\boldsymbol{u}_k$$

然后MPC的预测部分就开始了，

1. 递归推导与矩阵展开

在每个控制周期中，机器人未来的动力学方程写为离散线性时变系统（LTV）或分段线性系统：

$$x_{k+1} = A_k x_k + B_k u_k$$

其中：

- $x_k \in \mathbb{R}^{12}$ ：状态量（通常包含质心欧拉角、位置、角速度、线速度，重力项常以标量常数并入状态扩展或作为偏置）；
- $u_k \in \mathbb{R}^{12}$ （或 $3 \times \text{足端数}$ ）：控制量，即各足端的地面反作用力（GRF）。

从当前时刻状态 x_0 出发，逐步向前递推：

- $x_1 = A_0 x_0 + B_0 u_0$
- $x_2 = A_1 x_1 + B_1 u_1 = A_1 A_0 x_0 + A_1 B_0 u_0 + B_1 u_1$
- $x_3 = A_2 x_2 + B_2 u_2 = A_2 A_1 A_0 x_0 + A_2 A_1 B_0 u_0 + A_2 B_1 u_1 + B_2 u_2$
- ...

将预测视界 N 内的所有状态 $X = [x_1^T, x_2^T, \dots, x_N^T]^T$ 和控制输入 $U = [u_0^T, u_1^T, \dots, u_{N-1}^T]^T$ 堆叠，即可写成紧凑形式：

$$X = A_{\text{qp}} x_0 + B_{\text{qp}} U$$

矩阵结构

$$A_{\text{qp}} = \begin{bmatrix} A_0 \\ A_1 A_0 \\ A_2 A_1 A_0 \\ \vdots \\ \prod_{i=0}^{N-1} A_{N-1-i} \end{bmatrix}_{(N \cdot n_x) \times n_x}$$

$$B_{\text{qp}} = \begin{bmatrix} B_0 & 0 & 0 & \dots & 0 \\ A_1 B_0 & B_1 & 0 & \dots & 0 \\ A_2 A_1 B_0 & A_2 B_1 & B_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \left(\prod_{j=1}^{N-1} A_j \right) B_0 & \left(\prod_{j=2}^{N-1} A_j \right) B_1 & \dots & \dots & B_{N-1} \end{bmatrix}_{(N \cdot n_x) \times (N \cdot n_u)}$$

2. 标准二次规划（QP）形式转化

MPC 的原始跟踪目标函数为最小化状态偏差与控制能耗：

$$\min_{X,U} \sum_{k=1}^N (x_k - x_k^{\text{ref}})^T Q (x_k - x_k^{\text{ref}}) + \sum_{k=0}^{N-1} u_k^T R u_k$$

将其写为分块对角权重矩阵形式：

$$\min_{X,U} (X - X^{\text{ref}})^T \mathcal{Q} (X - X^{\text{ref}}) + U^T \mathcal{R} U$$

其中 $\mathcal{Q} = \text{diag}(Q, Q, \dots, Q)$, $\mathcal{R} = \text{diag}(R, R, \dots, R)$ 。

将 $X = A_{\text{qp}} x_0 + B_{\text{qp}} U$ 直接代入目标函数中消去 X ：

$$\begin{aligned} J(U) &= (A_{\text{qp}} x_0 + B_{\text{qp}} U - X^{\text{ref}})^T \mathcal{Q} (A_{\text{qp}} x_0 + B_{\text{qp}} U - X^{\text{ref}}) + U^T \mathcal{R} U \\ &= U^T (B_{\text{qp}}^T \mathcal{Q} B_{\text{qp}} + \mathcal{R}) U + 2(A_{\text{qp}} x_0 - X^{\text{ref}})^T \mathcal{Q} B_{\text{qp}} U + \text{const} \end{aligned}$$

整理为标准二次规划格式：

$$\min_U \frac{1}{2} U^T H U + g^T U$$

其中：

- **Hessian 矩阵**： $H = 2(B_{\text{qp}}^T \mathcal{Q} B_{\text{qp}} + \mathcal{R})$ （半正定/正定对称阵）
- **Gradient 向量**： $g = 2B_{\text{qp}}^T \mathcal{Q} (A_{\text{qp}} x_0 - X^{\text{ref}})$

但是需要注意的是，我们通过建模去根据控制输入U预测机器狗10个预测步的状态，但是我们只会将获得的控制输入用于一个预测周期，就可以理解成预测很多步，只取预测出的第一步。后续会马上预测新的N步，然后只取一步。正常来说上面已经推导出了H矩阵和g矩阵，直接扔进求解器求解就完事了，但是我们需要补充一下一些约束。

大家可以思考一下，我们的四足机器狗想要动起来是不是应该需要把腿给摆动起来，这样脱离地面的行为才能让狗动起来，然后地面的反作用力是给到了移动的动力。于是腿的运动就分成了接触相和摆动相两类，在摆动的过程中，不与地面接触，那么我们的足端接触力必然是为0的，也就是说我们强行

限制输入量 u 的部分为0，然后再进入求解器求解。这个就是mpc会根据当时所处的腿部的相位，也就是状态，以及之后 N 个周期的状态，然后去求解。我们要体现到约束的话也会很简单

四足机器人的物理约束主要施加在地面反作用力 u 上（摆动腿反力为0、单腿支撑力上限、地面摩擦锥）：

$$C_k u_k \leq d_k$$

前面有提到足端接触力上限的约束，我们只需要把接触相时的上限是正常一个数值，然后摆动相的力上限就为0，就能实现一个公式然后涵盖多种情况的表达。好的，那mpc需要的知道 N 个周期内的足端相位序列又是从哪里来呢，mit的处理就是设计了一个固定的步态周期，比如1条腿0.2s处于摆动状态，然后接着就是0.2s触地。总周期为0.4s一直循环下去。相当于是人为设定了一个开环周期。然后直接就把这些相位序列，比如触地就是1，摆动就是0，直接塞给mpc,让mpc去求解。

```
// ConvexMPCLocomotion.cpp 中的经典构建逻辑
for (int i = 0; i < horizonLength; i++) {
    int iter = iterationCounter + i * dtMPC / dtControl;
    gait->setIterations(horizonLength, iter);
    int* mpcTableCurrent = gait->getMpcTable();
    // mpcTable 包含未来 N 步内 4 条腿对应的 0（摆动）和 1（支撑）
    for (int leg = 0; leg < 4; leg++) {
        mpcTable[i * 4 + leg] = mpcTableCurrent[leg];
    }
}
```

现在来讲讲摆动相里面的内容，第一个部分就是落足点规划。

1. 落足点规划 (Footstep Planning)

落足点决定了机器人在下一个支撑相能够提供怎样的力矩来维持身体平衡。工程上最经典且广泛应用的是基于 **Raibert 启发式落足律 (Raibert Heuristic)** 的计算公式：

$$p_{\text{target}} = p_{\text{hip}} + p_{\text{symmetry}} + p_{\text{feedback}} + p_{\text{centrifugal}}$$

- **对称性落足项 (p_{symmetry})**：

保持匀速前进的基础步长，取支撑相时间 T_{stance} 的一半与期望速度的乘积：

$$p_{\text{symmetry}} = \frac{T_{\text{stance}}}{2} v_{\text{des}}$$

- **速度反馈调节项 (p_{feedback})**：

用于抵抗外界扰动。当实际速度 v_{act} 偏离期望速度时，落足点需向前/向后多迈一步以产生修正力矩：

$$p_{\text{feedback}} = k_{\text{feedback}}(v_{\text{act}} - v_{\text{des}})$$

其中增益常取倒立摆自然周期比例 $k_{\text{feedback}} \approx \sqrt{\frac{h}{g}}$ (h 为质心高度)。

- **离心力补偿项 ($p_{\text{centrifugal}}$)**：

在转弯绕圈时抵抗离心力：

$$p_{\text{centrifugal}} = \frac{1}{2} \sqrt{\frac{h}{g}} (\omega_{\text{yaw}} \times v_{\text{com}})$$

注意这个部分就是计算你这一轮摆动相最后落地的位置。

然后就是从抬脚点到落脚点的轨迹生成。

2. 空间足端轨迹生成 (Trajectory Generation)

获得起步点 p_{start} 、目标落足点 p_{target} 以及预设的抬腿高度 $h_{\text{clearance}}$ 后，需要生成一条平滑的 3D 轨迹 $p(t)$ ($t \in [0, T_{\text{swing}}]$)。

- **水平方向 (x, y 轴):**

从起点直接平滑插值到终点，常用**五次多项式 (Quintic Polynomial)** 或**三阶/五阶贝塞尔曲线 (Bézier Curve)**，确保起始和落地的速度、加速度均为 0 (或与机身速度匹配)，避免关节速度突变。

- **垂直方向 (z 轴):**

垂直方向需要先抬起再落下，通常在 $t = 0.5T_{\text{swing}}$ 时达到最高点 $z_{\text{max}} = z_0 + h_{\text{clearance}}$ 。常使用分段摆线 (Cycloid)、双三次样条或带中间控制点的贝塞尔曲线，保证离地 (Lift-off) 和触地 (Touch-down) 阶段法向速度受控。

在xy方向上能够直接用三次贝塞尔曲线

可以。MIT 这里虽然函数名叫 `cubicBezier()`，但它不是一般意义上“任意四个控制点”的三次贝塞尔，而是一个**特殊**的三次贝塞尔插值：

$$P_0 = P_1 = y_0$$

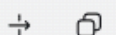
$$P_2 = P_3 = y_f$$

所以它实际上就是常见的 smoothstep：

$$3s^2 - 2s^3$$

官方 `Interpolation.h` 里直接写成：

```
</> C++
```



```
y_t yDiff = yf - y0;  
x_t bezier = x*x*x + 3*(x*x*(1-x));  
return y0 + bezier*yDiff;
```

把中间项整理一下：

$$s^3 + 3s^2(1-s) = 3s^2 - 2s^3$$

因此 MIT 实际使用的插值函数是：

$$y(s) = y_0 + (3s^2 - 2s^3)(y_f - y_0)$$

其中：

$$0 \leq s \leq 1$$

这就是源码中的 `cubicBezier()`。 GitHub

但是在z方向，也就是竖直方向，初始位置和末端位置都是0。直接插值没用，就选取了一个中间高度点，分两段做了三次贝塞尔插值。

$$z(s) = \begin{cases} z_0 + b(2s)h, & 0 \leq s < 0.5 \\ z_0 + h + b(2s - 1)(z_f - z_0 - h), & 0.5 \leq s \leq 1 \end{cases}$$

在摆动腿状态，上面已经设计出了足端的插值轨迹，我们就采用空间笛卡尔PD控制，先计算出控制足端需要的虚拟力，然后通过虚工原理求解到关节的力矩。

Step 1: 笛卡尔空间虚拟力（Virtual Force）计算

将足端建模为一个挂载在期望轨迹上的“三维弹簧-阻尼器”，产生的虚拟恢复力 $F \in \mathbb{R}^3$ 为：

$$F = K_p(p_{\text{des}} - p) + K_d(v_{\text{des}} - v)$$

- $K_p = \text{diag}(K_{px}, K_{py}, K_{pz})$ ：三维刚度矩阵（Stiffness），决定位置跟踪的硬度；
- $K_d = \text{diag}(K_{dx}, K_{dy}, K_{dz})$ ：三维阻尼矩阵（Damping），抑制轨迹振荡和超调。

Step 2: 虚功原理映射到关节空间力矩（Joint Torques）

根据刚体动力学中的虚功原理，末端力和关节力矩满足转置雅可比映射：

$$\tau_{\text{virtual}} = J^T(q)F = J^T(q)\left(K_p(p_{\text{des}} - p) + K_d(v_{\text{des}} - v)\right)$$

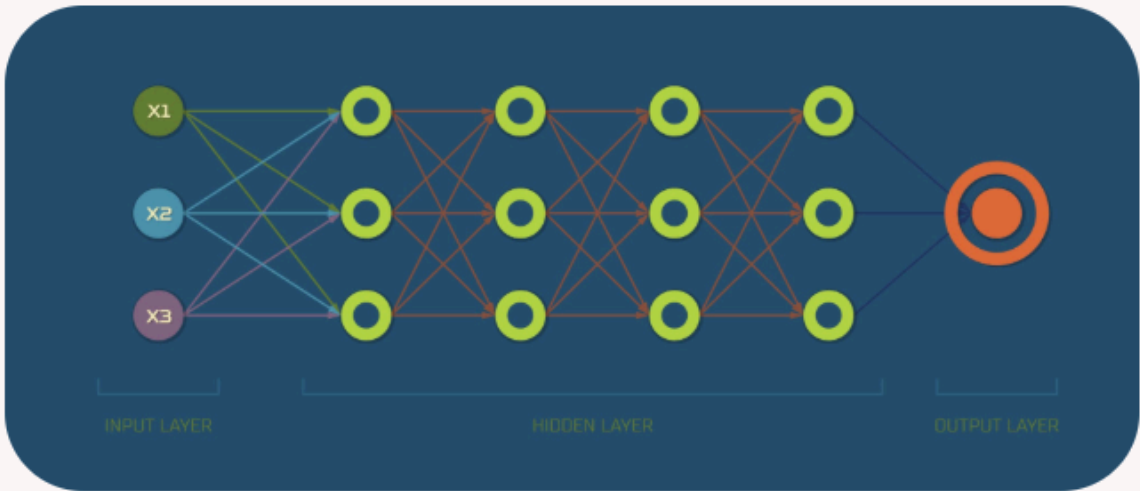
- $q \in \mathbb{R}^3$ ：该腿的 3 个关节角度（Hip, Thigh, Calf）；
- $J(q) = \frac{\partial p}{\partial q} \in \mathbb{R}^{3 \times 3}$ ：该腿足端相对于髌关节的几何雅可比矩阵（满足 $v = J(q)\dot{q}$ ）。

对于接触腿状态，足端接触力大小已经通过mpc求解得出，就省去了像摆动腿一样计算虚拟力的过程，而是直接由虚功原理，将力转化成关节力矩。

其实计算出足端接触力之后还有一种映射到关节的方式，就是wbc全身控制，这部分由于我本人没法很清晰地讲解出来，且上面的mpc已经能够具有极强的鲁棒性，就没在这里讲解，后续等大家入队探索。

强化学习常见知识及训练框架

MLP（Multilayer Perceptron，多层感知机）是一种最基础的前馈人工神经网络（Feedforward Neural Network）。它克服了早期单层感知机无法解决非线性问题（如异或问题 XOR）的缺陷，是现代深度学习的重要基石。



多层感知机网络结构. 来源: mastaka / Getty Images

核心网络结构

MLP 由多层节点（神经元）组成，通常采用全连接（Fully Connected, FC）形式，即上一层的每个神经元均与下一层的全部神经元相连：

- **输入层 (Input Layer)：** 接收原始特征数据，不进行计算操作，节点数等于特征维度。
- **隐藏层 (Hidden Layer)：** 位于输入与输出之间的一层或多层结构，负责特征的非线性提取与空间变换。
- **输出层 (Output Layer)：** 产生最终的预测结果，节点数和形式取决于任务类型（如回归任务通常输出连续值，分类任务常后接 Softmax 或 Sigmoid）。

Encoder（编码器） 的核心功能是将原始输入数据（如文本、图像、音频或传感器信号）转化为高度浓缩、富含语义特征的**隐空间表征（Latent Representation / Embedding）**。

Encoder 不是某种固定网络

下面这些其实都可以叫 Encoder：

$$o_{t-H:t} \rightarrow MLP \rightarrow z_t$$

也可以：

$$DepthImage_t \rightarrow CNN \rightarrow z_t$$

还可以：

$$o_{t-H:t} \rightarrow GRU \rightarrow z_t$$

所以Encoder 描述的是功能，不描述具体网络类型。

History Encoder从过去若干帧状态中提取环境/动力学信息，四足机器人里面一个比较典型的encoder应用就是

Latent / Latent Vector（隐的 / 潜在的）无法直接被观测到、但支配着显式观测数据的底层抽象特征。从encoder中提取出来的内部信息表示。

Decoder（解码器）的核心功能是从隐空间表征补全或者预测出完整的，具有实际意义的数据（文本，图像）

Estimator（估计器 / 状态估计网络）的核心功能是从不完整、带噪声或非直接相关的观测数据中，推断出无法直接测量（或直接测量成本极高）的底层物理量、潜在变量或统计参数。比如四足机器人的足端是否触地这个信息就很难在真机上面获取，但是可以通过状态估计网络

CNN（Convolutional Neural Network，卷积神经网络）专门擅长处理图像、深度图等空间信息的一种网络

RNN、LSTM 与 GRU 均属于循环神经网络（Recurrent Neural Network）家族。它们的核心作用是显式建模序列数据（Sequence Data）中的时序依赖关系与历史动态特征

经典 RNN（Vanilla RNN）

- **工作机制：**每个时间步 t 接收当前输入 x_t 和上一步的隐藏状态 h_{t-1} ，通过单层简单非线性变换更新隐藏状态：

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b)$$

VAE（Variational Autoencoder，变分自编码器）和编码器的作用类似，只不过是将输出的隐空间向量从原来的点分布变成了某种概率分布

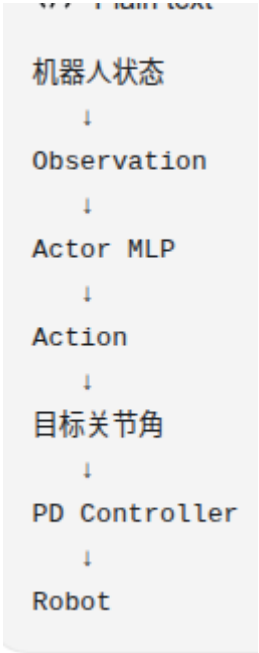
Teacher / Student Teacher 能看到更多Privileged Information(特权信息)比如地面的摩擦力，自己身体的线速度，以及周围的高度图，Student 通过一些方法学 Teacher 但只能用真机能提供的信息

Discriminator（判别器） 是对抗学习机制（如生成对抗网络 GAN、对抗模仿学习）中的核心组件，其本质是一个**二分类器或评分网络**。它的主要功能是**判断输入样本是来自真实数据分布（Real），还是由生成模型/策略伪造的（Fake）**。

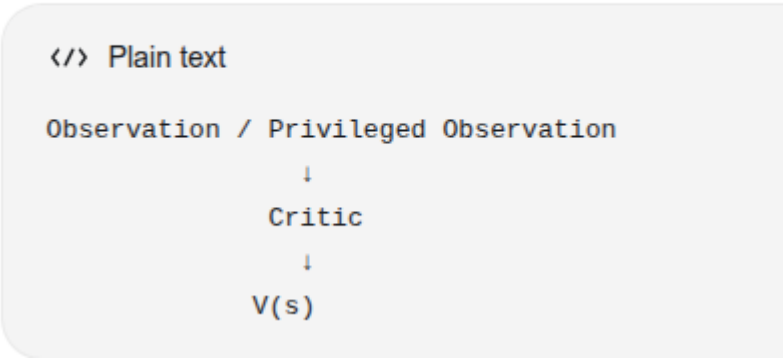
上面这些概念很多都是用的类似的网络，它们具体功能的区分主要是靠输入和输出是什么样的数据，尤其是这个输出会用来干什么或者怎么去处理这个输出。这才是决定了它的名称。比如actor critic encoder 它们使用的网络结构基本是MLP，但是它们的功能取决于 actor的输出就是机器人要采取的动作，critic的输出是给actor的评分，encoder的输出会作为actor的输入，所以就是编码器。

最经典的四足机器人训练框架

actor的作用



训练过程中另外存在：



Actor 的问题非常简单：

$$a_t = \pi(o_t)$$

也就是：

我现在看到机器人这个状态，我下一步应该给 12 个关节什么目标角度？

四足强化学习很多复杂论文，本质上都只是在改：

Policy 的输入到底应该是什么

以及：

怎么让 *Policy* 学到更好的内部表示

四足机器人常见的观测量会是关节角度，关节速度，重力在机身坐标系下的投影

Walk These Ways

这个论文和开源代码表达的主要内容是能不能根据额外的输入观测量的command表现出很多种不同的运动方式，它主要做了这两件事情

把“行为风格参数”作为策略输入。

同时设计和这些行为参数对应的奖励，让机器人真的按照这些参数去运动。

如果只有策略输入，而没有策略输入之后的相应奖励，网络也会忽视这些命令，比如我输入了一个更大步幅的参数，我就让更大步幅的奖励调整地更大。然后网络才会更倾向于更大的步幅。当网络意识到你把这个参数调大，就是想让它调大步幅的时候，就算实现这个框架的目的了。

我们训练一个四足机器人，如果给它一个移动的命令，通常都会是x,y,yaw这三维跟控制命令相关，

普通策略可能是：

$$(o_t, v_x^{cmd}, v_y^{cmd}, \omega_z^{cmd}) \rightarrow \pi \rightarrow a_t$$

WTW 可以抽象成：

$$(o_t, c_{velocity}, c_{gait}) \rightarrow \pi \rightarrow a_t$$

普通策略只告诉机器人做什么，WTW还告诉机器人用什么方式做。注意WTW还是会保留基本的控制命令。

论文的目标就是让相同任务，在不同 b_t 下拥有不同解法。作者把这叫做 Multiplicity of Behavior, MoB。

深入研究过训练代码就能发现机器人是在平地上训练的，它希望的就是最后训练出来的效果是可以通过我们自己去调整最后的策略输入量，选择我们想要的步态，步幅，步频这些，去适应相应的障碍，比如要上楼梯的话，我们就需要把步幅的参数调大，步频得稍小一点，调到最好的参数，那以后就用它来上楼梯。

回到训练的时候，

假设永远都是：

$$v_x^{cmd} = 1m/s$$

也就是说：

所有机器人都要 1 m/s 往前走。

传统 PPO 可能最终学出来：

</> Plain text

Policy A:

1 m/s forward

↓

固定偏好的 trot

但是 WTW 可以是：

</> Plain text

1 m/s forward

+

trot

+

3 Hz

+

抬腿 8 cm

+

正常机身高度

得到动作 A。

然后还是：

</> Plain text

1 m/s forward

+

trot

+

2 Hz

+

抬腿 20 cm

+

身体略低

得到动作 B。

4. WTW 原论文里的 behavior command 到底有哪些？

这个地方我建议你讲课时严格按照论文来，因为我上一条里提到“占空比”，其实那更接近后续代码实现里的 gait duration 概念；WTW 原论文正式定义的是 8 维 behavior vector。

原论文定义：

$$b_t = [\theta_1^{cmd}, \theta_2^{cmd}, \theta_3^{cmd}, f^{cmd}, h_z^{cmd}, \phi^{cmd}, s_y^{cmd}, h_{fz}^{cmd}]$$

分别是：  Proceedings of ... 

参数	实际意义
$\theta_1, \theta_2, \theta_3$	四条腿之间的相位关系
f	步频
h_z	机身高度
ϕ	机身 pitch
s_y	站立宽度
h_{fz}	摆动腿抬脚高度

11. 所以 WTW 真正核心是“command + command-conditioned reward”

例如你要求：

$$h_{fz}^{cmd} = 0.15m$$

训练时需要产生类似：

$$r_{footheight} = -(h_{fz} - h_{fz}^{cmd})^2$$

这样的约束。

要求：

$$h_z^{cmd}$$

就会有：

$$r_{bodyheight} = -(h_z - h_z^{cmd})^2$$

要求 body pitch：

$$r_{pitch} = -(\phi - \phi^{cmd})^2$$

代码里面还存在一个关键处理，就是内部存在时间相位信号，它会给每一只脚生成对应的相位变量，然后这个变量就要与我们输入的相位要求去做差求得reward

论文写的是类似：

$$t_t = [\sin(2\pi t_{FR}), \sin(2\pi t_{FL}), \sin(2\pi t_{RR}), \sin(2\pi t_{RL})]$$

这实际上是在告诉 Policy：

当前每条腿处于 gait cycle 的什么位置。

论文根据三个 phase offset 构造四条腿的相位，然后形成周期 timing reference。  Proceedings of ...

另外完整的WTW还包含了将完整的堆叠了30层的历史观测量输入到了策略。

Dreamwaq

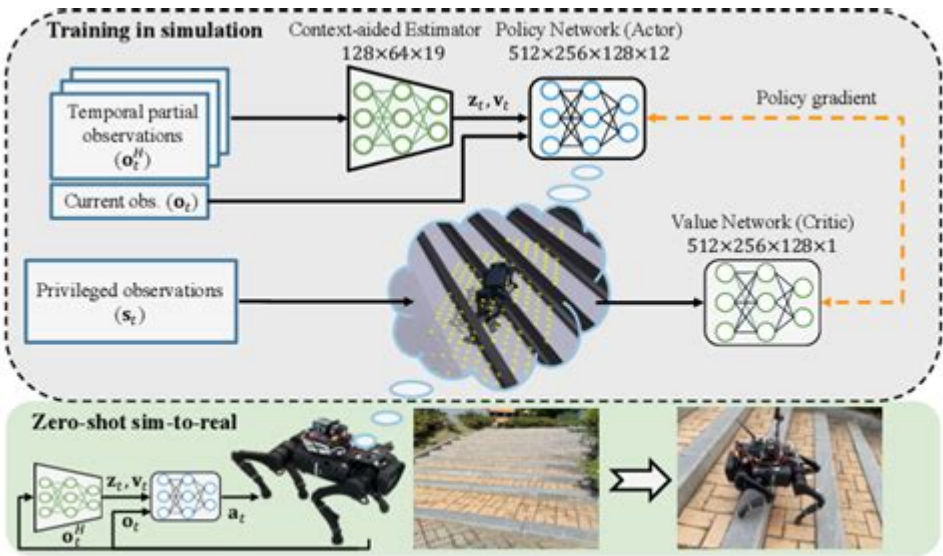
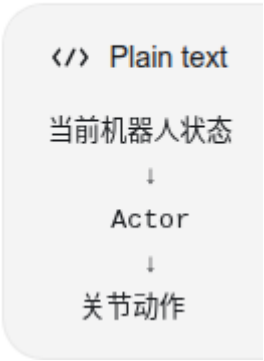
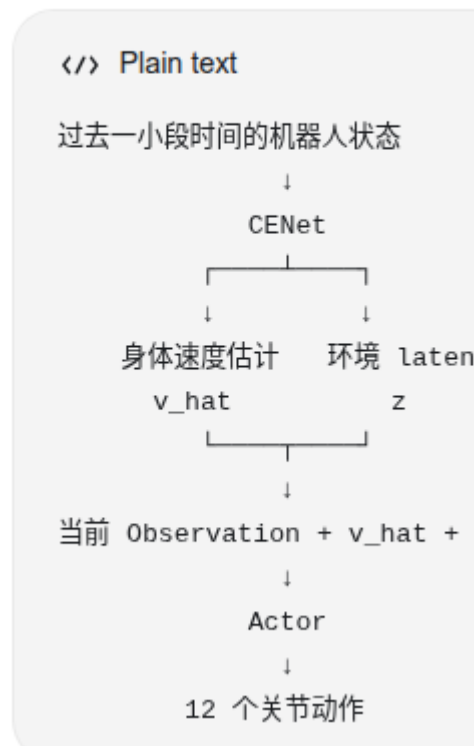


Fig. 1: Overview of DreamWaQ. By learning a locomotion policy in a

普通 PPO 大概是：



DreamWaQ 变成：



增加的是一个根据历史运动反应，推测当前身体状态和外部环境的网络。

Context-Aided Estimator Network（上下文辅助估计网络）

为什么单帧观测量不够，而要采用历史观测量，举一个例子，

论文定义：

$$o_t = [\omega_t, g_t, c_t, \theta_t, \dot{\theta}_t, a_{t-1}]$$

假设现在关节角，imu，速度，上一action也一样，但是机器人可能分别站在水泥地，沙地，草地，甚至是碎石坑里。单看一帧的观测量，是没办法推断出环境和身体速度的。上面这种定性的分析会经常用到强化学习里面网络框架的设计。需要大家培养出类似的直觉。

但是看历史观测量就不一样了，比如在遇到台阶和平地的区别，当腿正常摆动，也就是关节角度正常动，突然存在意外的响应，导致关节角度按照正常的走平地的方式无法运动，那就能够判断出是遇到一定高度的障碍物了。当然这里肯定还会结合到其他很多数据。

需要注意的是，CENet不只是一个"速度估计器"，

$$o_t^H \rightarrow CENet \rightarrow (\tilde{v}_t, z_t)$$

为什么还要专门估计身体速度，imu不是有加速度吗，积分不是可以直接得到速度吗？

我们前面在讲MPC在四足机器人中的运用时就已经提到过，实际imu会存在一个积分漂移的问题，时间一长误差就会累计。

但是四足机器人的运动控制对速度非常敏感，核心的reward就是速度跟踪。它一般会比其他reward参数大一个数量级。

怎么让这个网络估计速度能够更加贴和仿真器里面的实际速度呢呢，这里引入loss的概念，

Loss（损失 / 损失函数 / Loss Function） 在神经网络和机器学习中的核心作用，就是充当整个模型训练过程的“评估指挥官”与“梯度发动机”。

如果把神经网络比作一个在摸索中学习的学生，Loss 就是那张**考卷的扣分规则**：它量化了模型当前的输出与预期目标之间的差距，并通过数学机制引导模型参数该往什么方向调整。

用链式法则拆解更新过程

优化器要更新的是网络内部的参数 W ，而不是速度 v_t 。计算梯度时的前向与反向数据流如下：

1. 前向计算链路

$$o_t \xrightarrow{\text{估计器网络内部权重 } W} \tilde{v}_t \xrightarrow{\text{与真值 } v_t \text{ 作对比}} L_{\text{est}} = \frac{1}{d} \|\tilde{v}_t - v_t\|^2$$

2. 反向更新链路（链式法则求导）

根据微积分的链式法则，损失 L_{est} 对网络参数 W 的梯度计算为：

$$\frac{\partial L_{\text{est}}}{\partial W} = \underbrace{\frac{\partial L_{\text{est}}}{\partial \tilde{v}_t}}_{\text{预测误差项}} \cdot \underbrace{\frac{\partial \tilde{v}_t}{\partial W}}_{\text{网络结构对权重的导数}}$$

- 第一项 $\frac{\partial L_{\text{est}}}{\partial \tilde{v}_t} \propto (\tilde{v}_t - v_t)$ ：
这代表估计速度离真实速度偏了多少。如果当前估计的速度 $\tilde{v}_t$ 比真实速度 v_t 偏大，这一项就是正的；偏小就是负的。
- 第二项 $\frac{\partial \tilde{v}_t}{\partial W}$ ：
这代表估计器网络各层神经元对参数 W 的传递敏感度。

3. 参数 W 最终如何更新

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial L_{\text{est}}}{\partial W}$$

$L_{\text{est}} = \text{MSE}(\tilde{v}_t, v_t)$

比如我们设计这样去评估预测速度的损失。

这里MSE

MSE（Mean Squared Error，均方误差） 是统计学与机器学习中最基础且广泛使用的损失函数（Loss Function）或评估指标。

它通过计算**模型预测值与真实标签之间差值的平方和的均值**，来衡量预测值与真实值之间的偏离程度。

数学公式与梯度

对于包含 n 个样本的数据集，假设真实标签为 y_i ，模型预测值为 $\hat{y}_i$ ：

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

这部分就是显式状态估计了，什么是显式呢，它与后续会提到的隐式估计相对。显式估计的含义就是估计出来的数据带有明确物理意义的，可借助传感器验证的物理量，比如上面的速度，这些。而隐式就是不直接输出具有实际物理含义的物理量。

接下来就来讨论真正有意思的CENet估计出来的另外的16维的zt。

DreamWaQ 多做了一件事：

$$History \rightarrow z_t$$

但没有告诉网络：

</> Plain text

→

📄

```
z[0] = friction
z[1] = slope
z[2] = stair height
z[3] = payload
```

它没有这种明确监督。

这就是 **latent representation**。

例如真实世界状态可能包含：

$$x = [friction, terrain, payload, contact, disturbance, \dots]$$

DreamWaQ 不要求：

$$z = x$$

只要求：

| z 里面要留下对“预测机器人接下来会发生什么”有用的信息。

七、所以 DreamWaQ 用了一个很聪明的训练目标：预测下一帧

CENet 不只是估计当前速度。

它还要求：

$$z_t \rightarrow \text{Decoder} \rightarrow \tilde{o}_{t+1}$$

然后和真正的下一帧：

$$o_{t+1}$$

比较。

损失是：

$$MSE(\tilde{o}_{t+1}, o_{t+1})$$

训练目标就是需要对齐的参数，它的含义就是历史观测量通过CENet编码估计了一个隐空间向量，想要这个隐空间向量能够包含有对环境理解，那么我们就采用让一个decoder(解码器)去解析出来还未发生的下一帧观测量，需要注意的是这个decoder网络也需要更新，才能够预测地更准，但相应的，也只有隐空间向量里面包含了对环境的理解，这个decoder才能解码出来对吧。再来说一下为什么去估计下一帧的观测量能够体现对环境理解。假设在t时候，大家的action都是一致的，然后机器人要和环境去交互才能形成t+1时刻的状态(观测量)，不同环境下t+1观测量肯定不同，如果能够这个隐空间向量用来准确估计t+1观测量，那么它肯定就包含着环境信息。

八、为什么是 VAE，而不是普通 AutoEncoder？

普通 AutoEncoder 是：

$$History \rightarrow z \rightarrow o_{t+1}$$

DreamWaQ 使用的是：

$$\beta\text{-VAE}$$

也就是说 Encoder 不是直接给固定的：

$$z$$

而是学习一个概率分布：

$$q(z_t|o_t^H)$$

通常表示为：

$$\mu, \sigma$$

再通过重参数化采样 latent。

同时要求这个 distribution 不要乱跑，而是靠近：

$$N(0, I)$$

因此 VAE loss 是：

$$L_{\text{VAE}} = \text{MSE}(\tilde{o}_{t+1}, o_{t+1}) + \beta D_{KL}(q(z_t|o_t^H) \parallel N(0, I))$$

于是整个 CENet：

$$L_{\text{CENet}} = L_{\text{est}} + L_{\text{VAE}}$$

也就是：

$$L_{\text{CENet}} = \text{MSE}(\tilde{v}_t, v_t) + \text{MSE}(\tilde{o}_{t+1}, o_{t+1}) + \beta D_{KL}$$

这是 DreamWaQ 里面最核心的三个监督信号。  arXiv

1. 从“绝对的确定”到“带有不确定性的概率” (普通 AE vs. VAE)

普通 AutoEncoder 的死穴：离散与过拟合

普通的 AE 将一段历史观测序列 (History, 记作 o_t^H , 可能包含了过去几十帧的关节角度、IMU 读数等) 压缩成一个确定的隐向量 z 。

- 问题在于：真实世界的物理信号充满了噪声和不可预见性。如果采用普通 AE，输入稍微有一点传感器的抖动或未见过的地形冲击，映射出的 z 在隐空间中就会发生剧烈的跳变。这种缺乏平滑性的表征一旦喂给下游的策略网络 (Policy)，会导致机器人的动作输出极度抖动甚至崩溃。

VAE 的破局：用分布包容噪声

DreamWaQ 不再让 Encoder 输出一个死板的坐标 z ，而是输出一个高斯概率分布 $q(z_t|o_t^H)$ 。

- 网络实际预测的是分布的均值 μ (当前历史状态最可能对应的特征位置) 和标准差 σ (当前估计的不确定性)。
- 当系统面对未知的、复杂的动态历史时，它可以输出较大的 σ 来表示“我不太确定”，从而赋予下游决策系统极高的容错率和抗噪能力。

这里再讲一下什么是KL散度

KL 散度 (Kullback-Leibler Divergence)，又称相对熵 (Relative Entropy)，是信息论和概率统计中用于衡量两个概率分布之间差异程度的一种非对称度量指标。

这里讲一个针对于估计器一个常见的处理

1. 核心痛点：为什么需要 AdaBoot?

在采用 CENet (上下文编码器 / 估计器) 的强化学习框架中：

- 真实物理机 (部署时)：** 没有上帝视角的特权真值 (Ground Truth, 比如绝对精确的地面摩擦力)，Actor 只能 100% 依赖 CENet 给出的估计值 (Estimate) 来输出动作。
- 仿真训练早期：** 如果一开始就让 Actor 吃 CENet 的估计值，因为此时 CENet 还没训练好 (输出全是噪声)，Actor 本身也不会走路，这就变成了“瞎子指挥瞎子”，模型极难收敛。
- 仿真训练后期：** 如果一直让 Actor 吃仿真的特权真值，它会变成“温室里的花朵”，一旦部署到实机，面对 CENet 必然存在的估计误差，Actor 会瞬间无所适从导致摔倒 (即存在巨大的 Distribution Gap)。

AdaBoot 就是为了完美衔接这两个阶段而设计的**动态混合阀门**。

2. 数学机制解析：利用 Reward 衡量“成熟度”

AdaBoot 巧妙地利用了不同域随机化环境下的累积奖励变异系数来作为系统“成熟度”的指标，而不是简单地按训练步数线性递增。

第一步：计算奖励波动 $CV(R)$

$$CV(R) = \frac{\sigma_R}{\mu_R}$$

- μ_R 是各个随机化地形下机器人的平均奖励， σ_R 是奖励的标准差。
- **训练初期：** 机器人在某些简单地形碰巧走了两步（高 Reward），在复杂地形直接摔倒（低 Reward），方差 σ_R 极大，导致变异系数 $CV(R)$ 很大。这说明“Actor 还没有掌握通用的运动规律”。
- **训练后期：** 机器人在冰面、草地、负重下都能稳定行走，奖励值收敛且方差极小， $CV(R)$ 趋近于 0。这说明“Actor 已经具备了极强的基本功”。

第二步：映射为使用估计器的概率 p_{boot}

$$p_{boot} = 1 - \tanh(CV(R))$$

- 利用 $\tanh$ 函数将无穷大的波动系数平滑压缩到 $[0, 1)$ 之间。
- 当 $CV(R)$ 很大时（初期）， $\tanh$ 接近 1，导致 p_{boot} 接近 0。
- 当 $CV(R)$ 很小且趋近于 0 时（后期）， $\tanh$ 接近 0，导致 p_{boot} 接近 1。

具体怎么计算这个概率的可以简单看一下这个流程，要简单理解的话就是训练正常的话，就是一个非线性单调。

HIMloco

它的论文题目是 **Hybrid Internal Model: Learning Agile Legged Locomotion with Simulated Robot Response**, ICLR 2024。作者把地形、摩擦、外力等难以直接观测的环境因素统一看成“扰动”，不再要求网络显式估计这些扰动，而是让网络学习：

这些扰动作用到机器人之后，机器人自己表现出了什么响应。

然后从这个“机器人响应”里反推出当前该怎么控制。论文把这个内部表示称为 **Hybrid Internal Embedding**。

它整个网络框架和dreamwaq真的很像，但是在设定隐空间向量的训练目标时，不像原来的dreamwaq将隐空间向量解码成下一帧观测量。然后去贴近下一帧观测量。而是采用了将下一帧观测接入了一个名为目标编码器的网络，输出一个隐向量。现在就有了两个隐向量，一个是从历史状态中得到的，一个是从下一个状态的到的。论文后续做的工作就是让这两个编码网络对比纠正，让他们输出的隐空间向量都能完美地将自身受到环境的影响很好的分类出来。

CTS

在介绍CTS之前，先介绍一下传统的Teacher-Student 方案

在仿真训练四足机器人的时候，仿真环境能够给我们提供很多真机部署时根本拿不到的信息，比如高度图，就是获取机器人身体周围一个长方体大小内的地面高度信息。地面摩擦力，足端是否触地等信息。我们将它们归到privileged information里面，而真机能可靠获得到的主要是，imu,关节编码器，过去的action等本体感知。

这样就会产生一个矛盾，如果使用特权信息，机器狗能训的非常好，但是在真机部署的时候，我们又没办法提供特权信息。又想要机器狗能借助特权信息学的好，又想要能够部署，就有了传统 Teacher-Student 这个方案。

teacher能够获得到完整的特权信息，相当于就是在开卷考试，先把teacher用RL训练得很好。然后再冻结，也就是停止训练teacher。后续作为student学习的参考。

这个学习的过程可以通过监督学习，就是直接去比较类似观测量的情况下，学生和老师的action。

$$L_{BC} = \|a_t^{student} - a_t^{teacher}\|^2$$

或者让学生去模仿老师的隐空间向量，就是前面提到的dreamwaq一样，将历史观测量编码成的隐空间向量去贴近老师用特权信息编码而成的隐空间向量，这样学生的这个编码器就能学到老师对于环境的理解，也就能运用在真机部署上面

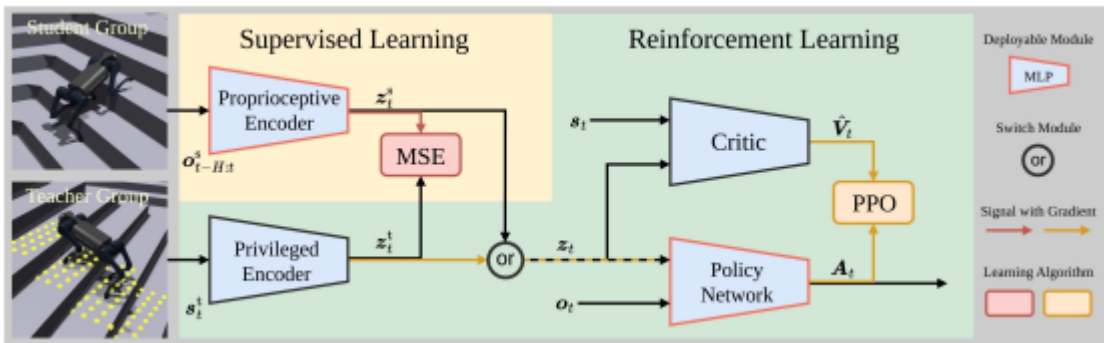
传统范式通常就是 **先 RL 训练 Teacher，再监督学习 Student**

CTS的改动第一点是Teacher 和 Student 同时存在

CTS 全称：

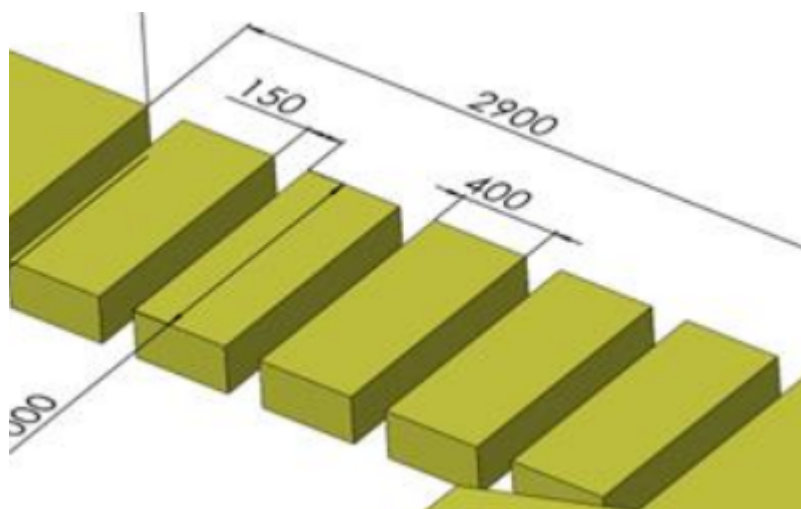
Concurrent(并发) Teacher-Student Reinforcement Learning

教师在学的时候，学生也在学，两边同时和环境交互，一起影响共享的策略



我们正常训练会开启很多并行环境，论文里面是用了8192个并行环境，然后教师和学生各占了4096个并行环境。学生将本体历史观测量编码成一个隐空间向量，然后老师将特权观测量编码成一个隐空间向量。然后用一个MSE，也就是方差去让学生的隐空间量贴近教师的隐空间量，这部分是监督学习的内容。学生和教师都生成了各自的隐空间向量，然后这个各自隐空间向量再结合本体观测量输入到共同的策略网络，然后输出动作。需要重点强调的是，训练收集数据过程中，学生和教师的数据是不会互相干扰的。然后绿色部分都是强化学习部分，教师的编码，学生进行的policy,教师进行的policy都是要经过强化学习。来分析一下为什么教师的编码器得到的隐向量能有用，大家应该还记得dreamwaq和himloco的隐向量都是通过类似对标下一帧观测量，让这个隐向量具有表征环境信息的能力。但是在CTS里面，教师隐向量的更新链路是靠强化学习，也就是这个编码器朝更能得分的方向更新。然后策略网络和critic也是朝着更能得分的方向更新。

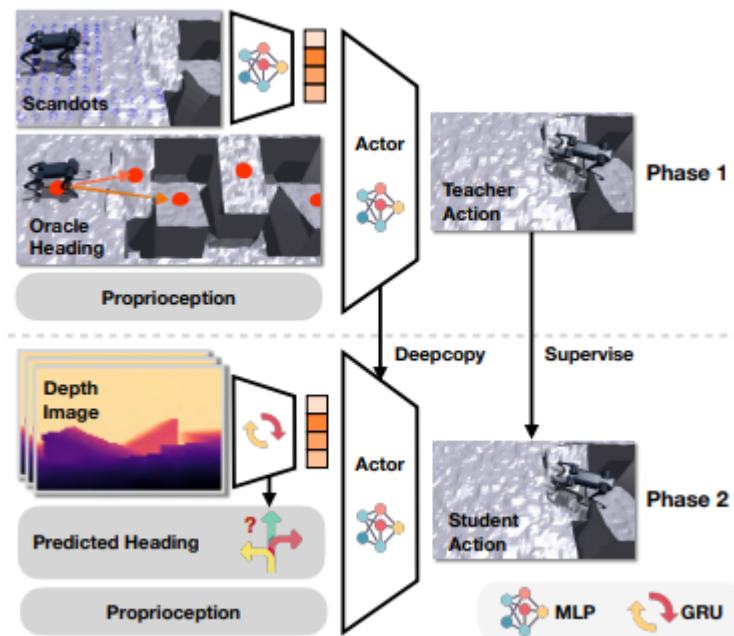
Extreme Parkour



可以想象一下当机器狗前方是30cm的沟壑，或者像图里面是一个间隔15cm的断桥，这种情况下机器狗靠本体感知是没法自动过去的。所以就必须引入外部感知，在extreme parkour则是使用了d435i深度相机获取深度信息进行训练，直接通过深度图和本体感知输出关节角命令，而不是先建图再规划这种传统视觉方案。

这个训练还是沿用了传统teacher_student框架

为什么不直接拿深度信息训练policy,是因为渲染深度信息是非常吃算力的。正常训练盲走可以开到4096个并行环境，但是如果渲染深度相机的话就会只能开64个并行环境左右。所以训练的数据量是大大减少的。很难才能收敛，当然如果计算资源足够的话，直接这样训，效果谁好也说不清。



scandots不是深度相机扫出来的点，而是直接从仿真环境里面直接获取到的机器狗周围规则范围内的地面高度。就是直接获得了一张局部地形高度描述。

障碍物朝向的问题，就是在常见的训练框架里面，速度指令都是x, y,yaw去随机采样的，训出来的东西我们就能用键盘或者手柄随意操作它，但是在本框架里面，因为有些地形是相当复杂的，人为很难去操作。所以设计的就是让策略自己去判断遇到一个障碍的时候怎么去调整它的yaw朝向。我们只需要给一个朝x的速度就行，然后它会自己越障。

所以在训练teacher的时候，就会针对障碍去人为设计红色必达点，然后计算红色必达点与此时机身的相对朝向。所以目标朝向也会作为观测量的一部分。

然后高程图的信息输入编码器，输出的隐空间向量也作为观测量的一部分。再加上本体感知，然后输入到策略，输出action

对于学生，获取到的深度图像信息是64x64，那么就会有4096维数据，输入网络维数太多，就经过CNN(卷积神经网络)

1. 第一棒：CNN（眼睛 —— 提取空间特征）

- 工作机制：当机载深度相机（如 D435i）拍下一张包含地形的高维深度图（例如 64×64 像素矩阵）时，直接把这 4096 个像素点喂给决策网络，网络会崩溃。CNN 通过多个“卷积核（Filters）”在图片上滑动，像扫描仪一样提取边缘、轮廓和深度梯度。
- 输出：它将一张庞大的图片，层层压缩、提纯，最终变成一个紧凑的一维特征向量（Feature Vector）。这个向量里浓缩了当前这一瞬间“前方有没有坑、台阶有多高”的核心信息。
- 致命弱点：CNN 只有“瞬间记忆”。它只能处理“这一帧”的图片。如果这一帧相机被遮挡，或者机器人抬头看天（FOV 丢失），CNN 提取出来的特征就是一片空白。

然后CNN只能得到某个时刻的视觉信息，还要体现之前像历史观测一样的效果还得要再加上一个GRU

2. 第二棒：GRU（短期记忆——处理时序序列）

为了解决 CNN 的“鱼的记忆”，我们将 CNN 输出的特征向量喂给下游的 GRU 网络。GRU 是 RNN（循环神经网络）的一种高级变体，专门用来处理随时间变化的序列数据。

它的内部有两个极其聪明的“门（Gates）”来管理记忆：

- **更新门（Update Gate）：决定“保留多少过去的记忆”。**
当机器人在远处看到一个台阶时，GRU 会把台阶的几何信息存入它的隐藏状态（Hidden State, h_t ）中。随着机器人靠近，更新门会决定将这个台阶的信息一直保留在脑海里，直到跨越动作发生。
- **重置门（Reset Gate）：决定“遗忘多少没用的信息”。**
当机器人成功跳上台阶，进入下一段平地时，重置门会被激活。它会果断清空之前关于那个台阶的记忆，防止机器人脑子里产生“前面还有台阶”的幻觉（这在代码中通常对应隐状态重置操作）。

它会随着策略的得分一同优化，直到能很好地判定什么是有用的信息该保留，什么是没用的信息该遗忘。

经过这两层 CNN 和 GRU，就会获得到隐空间向量，同时还会生成 yaw 的朝向角，逻辑就是通过看到障碍物，然后决定该朝哪然后去越过这个障碍。然后输入到 policy，最后输出 action。

感觉学生策略是靠什么去约束去更新呢，

用 Phase 1 teacher 产生的 action 作为标签，做 DAgger 式的行为蒸馏。

也就是说，视觉学生大概是：

$$D_t \rightarrow CNN \rightarrow f_t$$

然后：

$$(f_{t-k}, \dots, f_t) \rightarrow GRU \rightarrow z_t^{vision}$$

再把这个视觉 latent 和本体观测、heading 等一起送进 student policy：

$$(o_t, z_t^{vision}, \hat{\theta}_t) \rightarrow \pi_{student} \rightarrow a_t^{student}$$

teacher 则继续用 Phase 1 的 privileged exteroception，也就是 scandots：

$$(o_t, m_t, \theta_t^{oracle}) \rightarrow \pi_{teacher} \rightarrow a_t^{teacher}$$

训练时主要约束的是：

$$a_t^{student} \approx a_t^{teacher}$$

而不是：

$$z_t^{vision} \approx z_t^{teacher}$$

这大概就是所谓的 DAgger 式的行为蒸馏吧。

然后再来聊聊什么是 DAgger 式的行为蒸馏

1. 先讲最普通的 Supervised Imitation

假设 Teacher 已经很强了。

对于某个状态：

$$s_t$$

Teacher 给出动作：

$$a_t^T = \pi_T(s_t)$$

Student 看到自己允许看到的输入，比如：

$$o_t$$

然后输出：

$$a_t^S = \pi_S(o_t)$$

最直接的训练目标就是：

$$L_{\text{imit}} = \|a_t^S - a_t^T\|^2$$

中文解释就是：

Teacher 在这个状态输出什么动作，Student 就尽量输出一样的动作。

所以它本质上已经退化成一个普通监督学习问题：

</> Plain text

→

输入：

Student observation

标签：

Teacher action

网络：

Student Policy

目标：

预测 Teacher action

监督学习常用在错误不会累计的问题上，但机器人运动是一个连续过程，输出会影响与环境交互之后的状态，根据状态又会产生新的输出，就这样一直反复，如果中间产生一个错误，那么错误就会传递，会导致下次的动作更难，真机部署时可能会进入到从未训练过的状态。

5. DAgger 就是为了解决这个问题

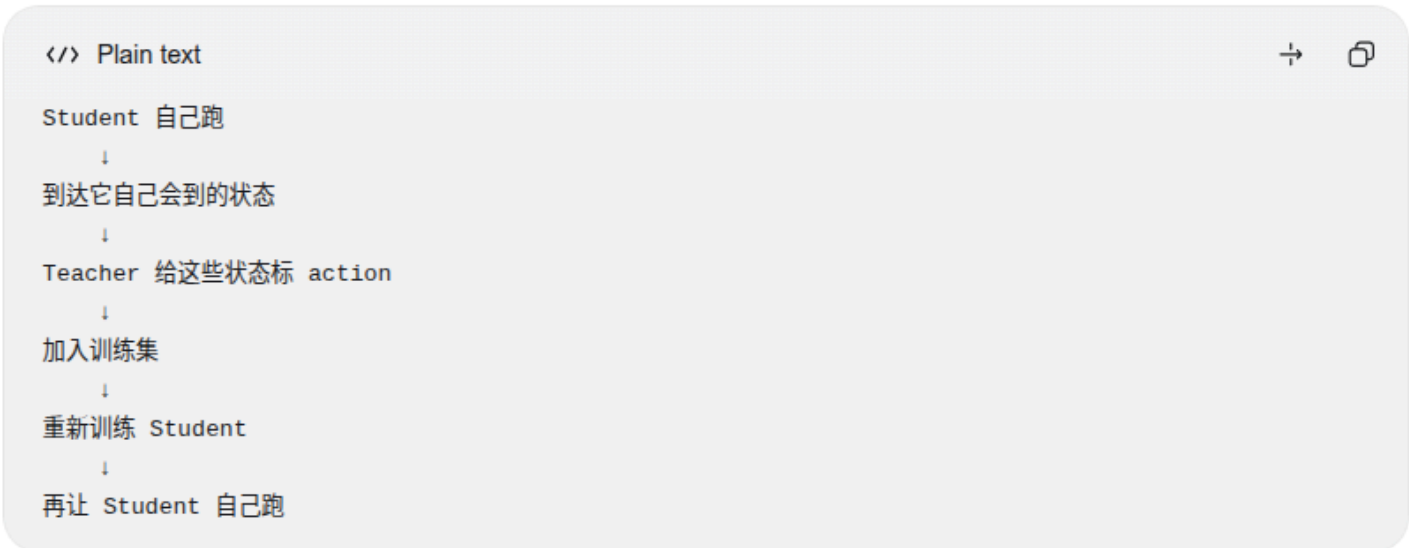
DAgger 全称：

Dataset Aggregation

核心思想非常简单：

不要只在 Teacher 的轨迹上训练 Student，而要让 Student 自己跑，然后在 Student 真正到达的状态上，请 Teacher 给正确答案。

所以流程变成：



第 i 轮 Student policy：

$$\pi_i$$

让它产生轨迹：

$$s_t \sim d_{\pi_i}$$

Teacher 给标签：

$$a_t^T = \pi_T(s_t)$$

加入数据集：

$$D_i = D_{i-1} \cup \{(s_t, a_t^T)\}$$

然后训练：

$$\pi_{i+1} = \arg \min_{\pi} \sum_{(s, a^T) \in D_i} L(\pi(s), a^T)$$

这就是最标准的 DAgger。

DAgger 就是让 Student 自己控制机器人，在 Student 实际遇到的状态下，让 Teacher 给出正确的动作作为标签，再把这些状态和对应的 Teacher 动作加入 Student 的训练数据集，通过模仿学习不断更

新 Student 的策略。

Clearance Reward

当落脚点距离 edge 小于约 5 cm 时惩罚：

$$r_{clearance} = - \sum_i c_i M[p_i]$$

其中 $M[p_i]$ 判断脚是不是踩在危险边缘附近。  ar5iv

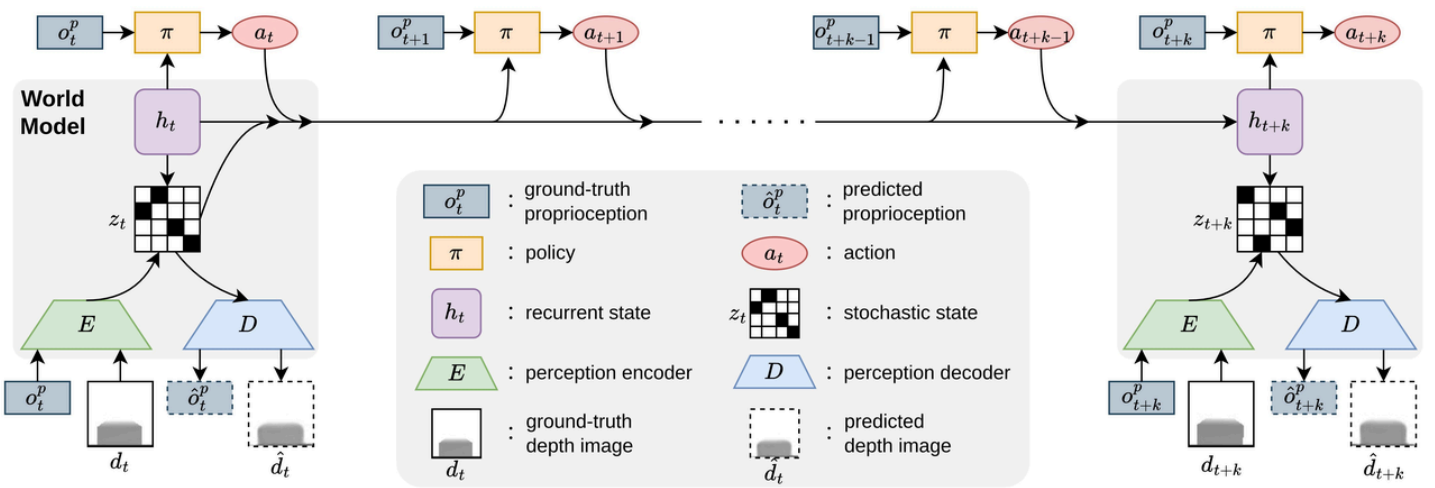
这一点特别适合给新人讲：

Sim2Real reward 不只是让机器人“完成任务”，还要主动给现实误差留下 margin。

说一些特殊的奖励设计，就是落脚点距离边缘小于5cm时的惩罚，这个边缘的界定是靠x方向是不是有急剧的落差。

然后原代码里面是会将四只脚的触地状态输入到观测量，作者用的是A1，有足端传感器，但是我们自己做的机器狗没有，我就专门加了一个状态估计器，将历史观测量编码成四维足端触地的0 或者1数据，然后和真实的触地状态做MSE去优化这个状态估计器。

WMP



WMP

WMP 全称是 World Model-based Perception，主要解决的是视觉四足运动里面，传统 Teacher-Student 框架存在的信息差问题。

前面讲 Extreme Parkour 的时候提到过，Teacher 在仿真里面可以直接读取地形高度点、障碍物参数等特权观测量，然后再让只能看到深度图的 Student 去模仿 Teacher。但是深度图和特权观测量提供的信息并不完全相同，所以 Student 很难百分之百复现 Teacher 的策略。并且像空中障碍、狭窄通道

这种地形，简单的地形高度点本身就无法完整表达，这时候连 Teacher 的输入都需要针对任务重新设计。

WMP 不再先训练 Teacher、再蒸馏 Student，而是在同一个训练阶段里面同时训练世界模型和运动策略。世界模型接收深度图、本体观测量以及机器人之前执行的动作，将这些高维信息压缩成一个隐空间状态。这个隐空间状态可以理解成机器人对当前环境的内部理解：前方是什么地形、自己刚才看到了什么、执行动作以后环境大概会怎样变化。

WMP 使用的是类似 Dreamer 的 RSSM，也就是循环状态空间模型。模型里面主要有一个确定性的历史状态 h 和一个随机隐变量 z 。编码器根据当前深度图和本体观测量得到后验隐变量；动态预测器不读取当前图像，只根据历史状态预测先验隐变量；解码器再尝试从隐变量中重建深度图和本体观测量。训练的时候，一方面要求重建结果尽量准确，另一方面使用 KL 散度让先验预测逐渐接近后验结果。这样就算某一时刻没有新的相机图像，模型也能根据之前看到的内容和机器人执行的动作，对当前环境做出估计。

这个历史状态非常重要。机器狗使用前视相机时，相机当前看到的是身体前面的地形；等机器人真正踩到这个位置时，这块地形可能已经处于相机视野之外了。因此策略不仅要看当前图像，还必须记住过去看到的内容。WMP 的循环状态相当于把历史深度图、本体状态和动作压缩成了一张动态更新的“脑内地图”，策略直接使用当前本体观测量和这个隐状态输出关节位置命令。

训练策略时仍然使用 PPO。Actor 的输入是本体观测量和世界模型的历史状态，Critic 在仿真训练阶段还可以额外使用接触力、地形点和随机化参数等特权信息。这里需要注意，策略输入的历史状态会进行 stop-gradient，也就是说策略的强化学习梯度不会反向破坏世界模型。WMP 也没有像传统 Model-Based RL 那样在世界模型里面生成虚拟轨迹来训练策略，因为仿真器本身已经可以高效地生成大量准确数据。它更主要的作用是做视觉信息压缩、历史记忆和环境预测。

考虑到深度图处理比较消耗算力，世界模型的运行频率比底层运动策略低。论文里面策略以 50Hz 输出动作，而世界模型每5个控制周期更新一次，也就是大约10Hz。两个世界模型更新时刻之间，策略继续使用最近一次得到的历史状态和实时本体观测量。这是在视觉响应速度和板载计算量之间做的折中。

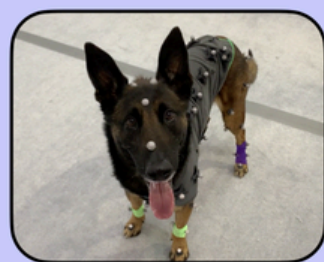
WMP 在斜坡、楼梯、沟壑、高台、狭窄通道和低矮障碍等地形上进行了训练。它的核心优势是不用为每一种地形单独设计特权观测量，也避免了 Teacher 到 Student 蒸馏时产生的性能损失。论文中的 A1 实机能够通过85cm的沟壑、爬上55cm的平台，并穿过高度约22cm的低矮通道。不过它仍然依赖深度相机质量和世界模型的预测能力；如果相机噪声、延迟或真实环境与仿真环境的差异太大，隐空间状态也可能产生错误判断。

另外，WMP 的训练奖励里面还使用了 AMP 风格奖励，使机器狗在完成越障任务的同时保持比较自然的运动方式。

原论文：[World Model-based Perception for Visual Legged Locomotion](#)

AMP

Train



Motion Capture
Data

Kinematic
Retargeting



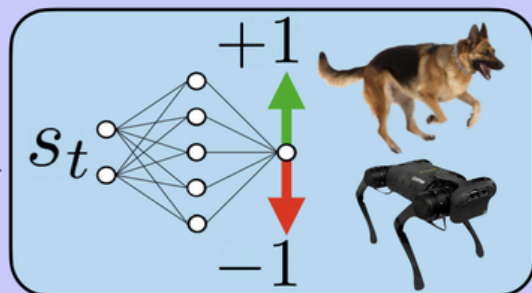
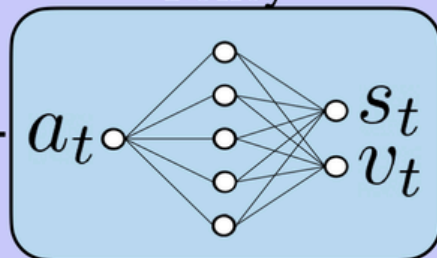
Task

Environment

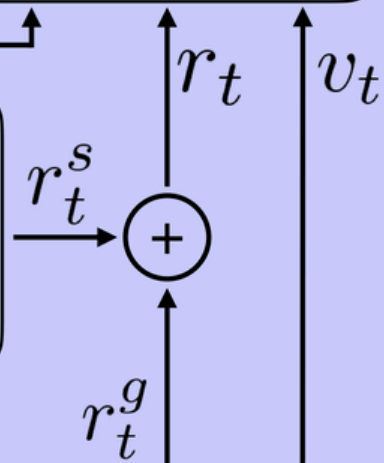


s_t

Policy



Motion Prior Reward



Deploy



AMP 全称是 Adversarial Motion Priors，也就是对抗运动先验。它主要解决的问题是：如果只给机器狗速度跟踪奖励，策略虽然可能学会按照目标速度移动，但也可能利用仿真器的漏洞，学出疯狂抖

腿、用很大的瞬时冲击力蹭着地面移动等动作。这些动作在仿真里面得分很高，放到实机上却不自然、耗能，甚至可能损坏电机。

传统做法是人为增加身体姿态、足端接触、关节加速度、力矩、动作平滑度等大量奖励和惩罚项。但是这些奖励需要反复调参，而且换一种机器人或者换一种任务以后，原来的参数不一定还能使用。AMP 的思路是，不再完全依靠人去定义“什么动作是自然的”，而是直接给网络看一段真实动物或者高质量控制器产生的参考动作，让网络自己学习动作的风格。

AMP 里面除了正常的策略网络，还增加了一个判别器。判别器会同时看到两类状态转移：一类来自参考动作数据，另一类来自当前策略控制机器人产生的动作。它的任务就是判断某一段状态变化到底是参考数据，还是策略自己生成的。

策略则要反过来尽可能“欺骗”判别器。如果策略生成的状态转移越来越像参考数据，判别器就会给出更高的风格奖励。因此 AMP 的训练方式和 GAN 有些类似：判别器不断学习区分参考动作和策略动作，策略不断学习生成更接近参考动作分布的运动。

机器人最终得到的奖励主要由两部分组成：

任务奖励负责告诉机器人要做什么，比如跟踪给定的前进速度、横向速度和转向速度。

风格奖励负责告诉机器人应该用什么样的动作完成任务，比如保持类似真实动物的步态、触地节奏和身体运动。

策略使用这两部分奖励通过 PPO 进行训练。这样既不会只顾着模仿动作而忘记控制命令，也不会为了完成控制命令而学出非常奇怪的姿态。

AMP 和严格的轨迹跟踪还有一个很重要的区别。轨迹跟踪要求机器人在某一个时刻对应参考动作里面的某一帧，而 AMP 只要求策略产生的状态转移在整体分布上像参考动作，并不要求逐帧完全一致。因此机器人可以根据速度命令和地形主动调整动作。比如参考数据是在平地上采集的，但策略仍然可以在粗糙地形中改变落脚位置，或者为了越过障碍暂时偏离参考动作。

四足机器人上的 AMP 工作使用了大约4.5秒的德国牧羊犬动作数据，其中包含踱步、快步、慢跑和原地转向等动作。经过动作重定向以后，将动物动作转换成 Unitree A1 可以执行的关节状态。最终训练出的策略会根据速度自动切换步态：低速时倾向于使用比较稳定的步态，高速时会切换到带腾空阶段的步态，而且能耗通常比复杂人工奖励训练出来的策略更低。

AMP 的优势是可以明显减少复杂奖励函数的设计工作，让动作更加自然、连续和节能，同时又保留完成新任务的自由度。不过它也有一些局限。最终学到的动作风格很依赖参考数据，如果数据质量差、动作种类太少或者动作重定向不合理，策略也会学到相应的问题。对抗训练本身也可能不稳定，判别器太强或者太弱都会影响策略学习。AMP 只能替代一部分风格奖励，关节限位、碰撞、力矩限制和实机安全等约束仍然需要单独处理。

简单来说，Walk These Ways 是人为输入步态、步频和抬腿高度等风格参数，再设计对应奖励；AMP 则是直接从参考动作数据里面学习“什么样的运动看起来合理”。两者都是在基本速度命令之外控制机器人的运动方式，只是获得运动风格的方法不同。